

CAHIER DES CHARGES

RAG Studio — Assistant Automobile Intelligent
Retrieval-Augmented Generation avec Base de Données

Projet	RAG Studio — Car Assistant
Version	2.0
Date	06/05/2026
Framework Backend	FastAPI (Python)
LLM	Ollama / llama3.2 (local)
Source de données	Base de données relationnelle (MySQL / PostgreSQL)
Statut	En cours de développement

1. Présentation du Projet

RAG Studio est une application web de question-réponse intelligente basée sur l'architecture RAG (Retrieval-Augmented Generation). Dans sa version 2.0, le projet évolue vers un assistant spécialisé dans le domaine automobile, exploitant une base de données relationnelle comme source de connaissance principale, en remplacement de l'indexation de texte brut ou de pages web.

Le système permet à un utilisateur de poser des questions en langage naturel sur des véhicules, et d'obtenir des réponses précises générées par un modèle de langage local (Ollama / llama3.2), enrichies par les données extraites dynamiquement depuis la base de données.

2. Objectifs

2.1 Objectifs fonctionnels

- Permettre à l'utilisateur de poser des questions en langage naturel sur des voitures.
- Interroger dynamiquement la base de données selon la question posée.
- Construire un contexte pertinent à partir des données récupérées.
- Générer une réponse cohérente via le LLM local (llama3.2).
- Afficher les sources de données utilisées pour chaque réponse.
- Supporter le streaming de la réponse token par token (SSE).

2.2 Objectifs non-fonctionnels

- Performance : réponses en moins de 10 secondes pour des requêtes standards.
- Confidentialité : toutes les données et le LLM restent en local.
- Maintenabilité : architecture modulaire et code documenté.
- Extensibilité : ajout facile de nouvelles tables ou entités automobiles.
- Disponibilité : service accessible 24/7 en environnement local.

3. Périmètre et Cas d'Usage

3.1 Domaine métier

Le projet couvre le domaine automobile avec les entités suivantes stockées en base de données :

- Véhicules : marque, modèle, année, motorisation, transmission, prix
- Caractéristiques techniques : puissance, couple, consommation, émissions CO2
- Équipements & options : sécurité active, confort, multimédia
- Avis et évaluations : notes, commentaires utilisateurs
- Stock / disponibilité : quantité, délai de livraison, concessionnaires

3.2 Exemples de questions supportées

Question utilisateur	Traitement attendu
Quelles voitures coûtent moins de 25 000 € ?	Requête SQL sur la table véhicules (prix)
Compare la Toyota Yaris et la Peugeot 208.	Jointure multi-lignes + synthèse LLM
Quelle est la consommation de la Golf 8 ?	Filtrage par modèle, extraction specs
Quels SUV hybrides sont disponibles ?	Filtrage motorisation + type + stock
Donne-moi les meilleures voitures notées.	Tri par note décroissante

4. Architecture Technique

4.1 Vue d'ensemble

L'architecture RAG est adaptée pour fonctionner avec une base de données relationnelle au lieu d'une base vectorielle. Le pipeline se déroule en 4 étapes :

Étape	Composant	Description
1 — Réception	FastAPI /api/query	Réception de la question utilisateur
2 — Traduction SQL	Query Translator	Conversion NL → SQL via règles ou LLM
3 — Récupération	Database Connector	Exécution SQL, formatage des résultats en texte
4 — Génération	Ollama llama3.2	Prompt enrichi → génération de la réponse

4.2 Stack technologique

Couche	Technologie	Rôle
Backend	FastAPI (Python 3.11+)	API REST + SSE streaming
LLM	Ollama + llama3.2	Génération de réponses (local)
Base de données	MySQL / PostgreSQL	Source de données automobile
ORM / Driver	SQLAlchemy / asyncpg	Connexion et requêtage async
Frontend	HTML / JS (Jinja2)	Interface utilisateur
Similarité	TF-IDF (cosine sim)	Ranking des résultats DB

5. Spécifications des Endpoints API

Méthode	Route	Description
GET	/	Interface web principale (HTML)
POST	/api/index	SUPPRIMÉ — remplacé par la connexion DB
POST	/api/query	Question NL → SQL → LLM → réponse JSON
POST	/api/query/stream	Même pipeline en streaming SSE
GET	/api/status	Vérification connexion Ollama + DB
GET	/api/chunks	Retourne les résultats DB formatés
POST	/api/db/connect	Configure la connexion à la base de données
GET	/api/db/schema	Retourne le schéma des tables disponibles

6. Modèle de Données

La base de données automobile comprend les tables principales suivantes :

Table : vehicles

Colonnes : id | brand | model | year | price | fuel_type | transmission

Table : specs

Colonnes : id | vehicle_id (FK) | horsepower | torque | consumption_L100 | co2_g_km

Table : equipment

Colonnes : id | vehicle_id (FK) | feature_name | category | standard

Table : reviews

Colonnes : id | vehicle_id (FK) | rating | comment | created_at

Table : stock

Colonnes : id | vehicle_id (FK) | dealer_id | quantity | delivery_weeks

7. Évolutions par rapport à la Version 1.0

Fonctionnalité	Version 1.0	Version 2.0
Source de données	URL / Texte brut	Base de données relationnelle
Indexation	TF-IDF en mémoire	Requêtes SQL dynamiques
Endpoint /api/index	Obligatoire	Supprimé / remplacé
Domaine	Générique	Spécialisé automobile
Connexion externe	Web scraping (httpx)	Connecteur DB (SQLAlchemy)
Schéma des données	Chunks de texte	Tables relationnelles structurées
Nouveaux endpoints	—	/api/db/connect, /api/db/schema

8. Contraintes et Exigences

8.1 Contraintes techniques

- Ollama doit être installé et en cours d'exécution sur localhost:11434.
- Le modèle llama3.2 doit être téléchargé via 'ollama pull llama3.2'.
- Python 3.11+ requis avec les dépendances FastAPI, httpx, SQLAlchemy, BeautifulSoup4.
- La base de données doit être accessible depuis le serveur FastAPI.
- OLLAMA_ORIGINS='*' doit être configuré pour les requêtes cross-origin.

8.2 Contraintes de sécurité

- Toutes les requêtes SQL doivent être paramétrées (protection injection SQL).
- Les credentials de la base de données ne doivent pas être exposés côté client.
- Le LLM fonctionne entièrement en local (aucune donnée envoyée vers des services cloud).

9. Planning de Développement

Phase	Tâches	Durée estimée
Phase 1 — Base	Modélisation DB, schéma SQL, données de test	1 semaine
Phase 2 — Backend	Connecteur DB, query translator, endpoint /api/query v2	2 semaines
Phase 3 — RAG DB	Intégration TF-IDF sur résultats SQL, prompt engineering	1 semaine
Phase 4 — Frontend	UI mise à jour, affichage sources DB, formulaire connexion	1 semaine
Phase 5 — Tests	Tests unitaires, tests d'intégration, validation QA	1 semaine
Phase 6 — Livraison	Documentation, déploiement, démonstration	3 jours

10. Livrables Attendus

- Code source complet du backend FastAPI (main.py, db_connector.py, query_translator.py)
 - Scripts SQL de création et de peuplement de la base de données automobile
 - Interface web fonctionnelle (templates HTML + static JS/CSS)
 - Documentation technique (README.md) avec instructions d'installation
 - Ce cahier des charges (PDF)
 - Rapport de tests et résultats de validation
-